

Singly Circular Linked List

1.Insert at Beginning.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node* next;
7  };
8
9  struct Node* head = NULL;
10
11 void insertatBegin(int value) {
12     struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));
13     newnode->data = value;
14
15     if (head == NULL) {
16         newnode->next = newnode;
17         head = newnode;
18     }
19     else {
20         struct Node* temp = head;
21
22         while (temp->next != head) {
23             temp = temp->next;
24         }
25
26         newnode->next = head;
27         temp->next = newnode;
28
29         head = newnode;
30     }
31 }
32 void display() {
33     if (head == NULL) {
34         printf("List is empty\n");
35         return;
36     }
37
38     struct Node* temp = head;
39
40     do {
41         printf("%d -> ", temp->data);
42         temp = temp->next;
43     } while (temp != head);
44
45     printf("(back to head)\n");
46 }
47
48 int main() {
49     insertatBegin(11);
50     insertatBegin(65);
51     insertatBegin(115);
52
53     display();
54 }
```

115 -> 65 -> 11 -> (back to head)

=== Code Execution Successful ===

115 -> 65 -> 11 -> (back to head)

=== Code Execution Successful ===

```
37
38     struct Node* temp = head;
39
40     do {
41         printf("%d -> ", temp->data);
42         temp = temp->next;
43     } while (temp != head);
44
45     printf("(back to head)\n");
46 }
47
48 int main() {
49     insertatBegin(11);
50     insertatBegin(65);
51     insertatBegin(115);
52
53     display();
54
55     return 0;
56 }
```

115 -> 65 -> 11 -> (back to head)

=== Code Execution Successful ===

2.Insert at End.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void display()
13 {
14     struct node *temp = head;
15
16     if(temp == NULL)
17     {
18         printf("List is empty\n");
19         return;
20     }
21
22     while(temp != NULL)
23     {
24         printf("%d -> ", temp->data);
25         temp = temp->next;
26     }
27     printf("NULL\n");
```

```
10 -> 20 -> 30 -> NULL
10 -> 20 -> 30 -> 40 -> NULL
```

=== Code Execution Successful ===

```
28 }
29
30 void insertatend(int value)
31 {
32     struct node *newnode, *temp;
33
34     newnode = (struct node*)malloc(sizeof(struct node));
35     newnode->data = value;
36     newnode->next = NULL;
37
38     if(head == NULL)
39     {
40         head = newnode;
41     }
42     else
43     {
44         temp = head;
45         while(temp->next != NULL)
46         {
47             temp = temp->next;
48         }
49         temp->next = newnode;
50     }
51 }
52
53 int main()
54 {
55     insertatend(10);
```

```
10 -> 20 -> 30 -> NULL
10 -> 20 -> 30 -> 40 -> NULL
```

=== Code Execution Successful ===

```
48     }
49     temp->next = newnode;
50 }
51 }
52
53 int main()
54 {
55     insertatend(10);
56     insertatend(20);
57     insertatend(30);
58
59
60     display();
61
62     insertatend(40);
63
64     display();
65
66     return 0;
67 }
```

```
10 -> 20 -> 30 -> NULL
10 -> 20 -> 30 -> 40 -> NULL
```

```
=== Code Execution Successful ===
```

3. Insertion at Position.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void display()
13 {
14     struct node *temp = head;
15     while(temp != NULL)
16     {
17         printf("%d -> ", temp->data);
18         temp = temp->next;
19     }
20     printf("NULL\n");
21 }
22
23 void insertatposition(int value, int pos)
24 {
25     struct node *newnode, *temp;
26     int i;
27
28     newnode = (struct node*)malloc(sizeof(struct node));
29     newnode->data = value;
30
31     if(pos == 1)
32     {
33         newnode->next = head;
34         head = newnode;
35     }
36     else
37     {
38         temp = head;
39         for(i = 1; i < pos-1; i++)
40         {
41             temp = temp->next;
42         }
43
44         newnode->next = temp->next;
45         temp->next = newnode;
46     }
47 }
48
49 int main()
50 {
51     insertatposition(10,1);
52     insertatposition(20,2);
53     insertatposition(30,3);
54
55     printf("Before insertion:\n");
```

Before insertion:
10 -> 20 -> 30 -> NULL
After insertion at position 3:
10 -> 20 -> 25 -> 30 -> NULL
=== Code Execution Successful ===

```
28     newnode = (struct node*)malloc(sizeof(struct node));
29     newnode->data = value;
30
31     if(pos == 1)
32     {
33         newnode->next = head;
34         head = newnode;
35     }
36     else
37     {
38         temp = head;
39         for(i = 1; i < pos-1; i++)
40         {
41             temp = temp->next;
42         }
43
44         newnode->next = temp->next;
45         temp->next = newnode;
46     }
47 }
48
49 int main()
50 {
51     insertatposition(10,1);
52     insertatposition(20,2);
53     insertatposition(30,3);
54
55     printf("Before insertion:\n");
```

Before insertion:
10 -> 20 -> 30 -> NULL
After insertion at position 3:
10 -> 20 -> 25 -> 30 -> NULL
=== Code Execution Successful ===

```

45     temp->next = newnode;
46 }
47 }
48
49 int main()
50 {
51     insertatposition(10,1);
52     insertatposition(20,2);
53     insertatposition(30,3);
54
55     printf("Before insertion:\n");
56     display();
57
58     insertatposition(25,3);
59
60     printf("After insertion at position 3:\n");
61     display();
62
63     return 0;
64 }

```

```

^ Before insertion:
10 -> 20 -> 30 -> NULL
After insertion at position 3:
10 -> 20 -> 25 -> 30 -> NULL

```

```

=== Code Execution Successful ===

```

4. Deletion at Beginning.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void display()
13 {
14     struct node *temp = head;
15
16     while(temp != NULL)
17     {
18         printf("%d -> ", temp->data);
19         temp = temp->next;
20     }
21     printf("NULL\n");
22 }
23
24 void deleteatbeginning()
25 {
26     struct node *temp;
```

```
10 -> 20 -> 30 -> NULL
20 -> 30 -> NULL
```

=== Code Execution Successful ===

```
28     if(head == NULL)
29     {
30         printf("List is empty\n");
31     }
32     else
33     {
34         temp = head;
35         head = head->next;
36         free(temp);
37     }
38 }
39
40 int main()
41 {
42     struct node *n1,*n2,*n3;
43
44     n1 = (struct node*)malloc(sizeof(struct node));
45     n2 = (struct node*)malloc(sizeof(struct node));
46     n3 = (struct node*)malloc(sizeof(struct node));
47
48     n1->data = 10;
49     n1->next = n2;
50
51     n2->data = 20;
52     n2->next = n3;
53
54     n3->data = 30;
```

```
10 -> 20 -> 30 -> NULL
20 -> 30 -> NULL
```

=== Code Execution Successful ===

```
47
48     n1->data = 10;
49     n1->next = n2;
50
51     n2->data = 20;
52     n2->next = n3;
53
54     n3->data = 30;
55     n3->next = NULL;
56
57     head = n1;
58
59     display();
60
61     deleteatbeginning();
62
63     display();
64
65     return 0;
66 }
```

```
10 -> 20 -> 30 -> NULL
20 -> 30 -> NULL
```

```
=== Code Execution Successful ===
```


5. Deletion at End.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void display()
13 {
14     struct node *temp = head;
15
16     while(temp != NULL)
17     {
18         printf("%d -> ", temp->data);
19         temp = temp->next;
20     }
21     printf("NULL\n");
22 }
23
24 void deleteatend()
25 {
26     struct node *temp, *prev;
27
```

```
10 -> 20 -> 30 -> NULL
10 -> 20 -> NULL
```

=== Code Execution Successful ===

```
28     if(head == NULL)
29     {
30         printf("List is empty\n");
31     }
32     else if(head->next == NULL)
33     {
34         free(head);
35         head = NULL;
36     }
37     else
38     {
39         temp = head;
40         while(temp->next != NULL)
41         {
42             prev = temp;
43             temp = temp->next;
44         }
45         prev->next = NULL;
46         free(temp);
47     }
48 }
49
50 int main()
51 {
52     struct node *n1,*n2,*n3;
53
54     n1 = (struct node*)malloc(sizeof(struct node));
55     n2 = (struct node*)malloc(sizeof(struct node));
```

```
10 -> 20 -> 30 -> NULL
10 -> 20 -> NULL
```

=== Code Execution Successful ===

```
56     n3 = (struct node*)malloc(sizeof(struct node));
57
58     n1->data = 10;
59     n1->next = n2;
60
61     n2->data = 20;
62     n2->next = n3;
63
64     n3->data = 30;
65     n3->next = NULL;
66
67     head = n1;
68
69     display();
70
71     deleteatend();
72
73     display();
74     return 0;
75 }
```

```
10 -> 20 -> 30 -> NULL
10 -> 20 -> NULL
```

=== Code Execution Success ===

6. Deletion at Position.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void display()
13 {
14     struct node *temp = head;
15
16     while(temp != NULL)
17     {
18         printf("%d -> ", temp->data);
19         temp = temp->next;
20     }
21     printf("NULL\n");
22 }
23
24 void deleteatposition(int pos)
25 {
26     struct node *temp = head, *prev;
27     int i;
```

```
Before deletion:
101 -> 102 -> 103 -> 104 -> NULL
After deletion at position 3:
101 -> 102 -> 104 -> NULL
```

=== Code Execution Successful ===

```
29     if(head == NULL)
30     {
31         printf("List is empty\n");
32         return;
33     }
34
35     if(pos == 1)
36     {
37         head = temp->next;
38         free(temp);
39     }
40     else
41     {
42         for(i = 1; i < pos; i++)
43         {
44             prev = temp;
45             temp = temp->next;
46         }
47
48         prev->next = temp->next;
49         free(temp);
50     }
51 }
52
53 int main()
54 {
55     struct node *n1,*n2,*n3,*n4;
```

```
Before deletion:
101 -> 102 -> 103 -> 104 -> NULL
After deletion at position 3:
101 -> 102 -> 104 -> NULL
```

=== Code Execution Successful ===

```

58     n2 = (struct node*)malloc(sizeof(struct node));
59     n3 = (struct node*)malloc(sizeof(struct node));
60     n4 = (struct node*)malloc(sizeof(struct node));
61
62     n1->data = 101;
63     n1->next = n2;
64
65     n2->data = 102;
66     n2->next = n3;
67
68     n3->data = 103;
69     n3->next = n4;
70
71     n4->data = 104;
72     n4->next = NULL;
73
74     head = n1;
75
76     printf("Before deletion:\n");
77     display();
78
79     deleteatposition(3);
80
81     printf("After deletion at position 3:\n");
82     display();
83
84     return 0;
85 }

```

```

^
Before deletion:
101 -> 102 -> 103 -> 104 -> NULL
After deletion at position 3:
101 -> 102 -> 104 -> NULL

```

```

=== Code Execution Successful ===

```